

Inventory Management Tool

In-Memory Advanced Data Structures & Collection Optimization

TRAINEE: Benatic Rithan B

INSTITUTION: KIT, Coimbatore

DATE: 15 June 2026

MENTOR: Rathees G

DEPARTMENT: CSE

STATUS: Completed Successfully

1. Executive Summary

This technical report documents the implementation of an high-performance, in-memory **Inventory Management Tool** developed during Day 6 of the Python Backend Development Internship. The design objectives focused on selecting and combining core Python collection types (Lists, Tuples, Sets, and Dictionaries) based on their runtime complexity constraints and mutability models. The engineered application implements robust real-time metrics tracking, stock mutations, unique category tracking, and transactional histories while establishing efficient item lookup routines.

2. Data Structure Selection & Complexity Architecture

To eliminate processing bottlenecks during high-frequency transaction processing, primitive data structures were assigned to isolated functional roles based on asymptotic execution behaviors:

- **Dictionaries (Hash Maps):** Chosen as the primary product catalog storage. Maps unique keys (`sku_id`) directly to complex attribute sets to achieve constant time lookup complexities, **$O(1)$** .
- **Tuples (Immutable Sequences):** Enforces read-only database structures for permanent transactional stamps (e.g., timestamp, identifier, quantity delta) protecting audit entities from accidental in-flight alteration.
- **Lists (Dynamic Arrays):** Utilized as sequential, mutable arrays to accumulate ongoing historical transactions.
- **Sets (Hash Tables):** Applied to maintain unique strings of catalog cross-references and supplier sectors, ensuring instant deduplication processing at **$O(1)$** intersection checkpoints.

3. Structural Mapping Matrix

Collection Type	System Implementation Role	Core Structural Feature	Primary Operational Advantage
Lists	Audit / Transaction Ledger	Ordered & Mutable Array	Efficient append actions for chronological tracking.
Tuples	Transaction Records	Ordered & Immutable Segment	Guarantees data integrity for committed ledgers.
Sets	Category Cataloging	Unordered & Unique Nodes	Instant deduplication and quick inclusion tests.
Dictionaries	Live Product Database	Key-Value Association Map	O(1) lookups by unique product identifiers.

4. Technical Code Implementation

The code segment below showcases the integration of multiple structures to coordinate state updates safely:

```
class InventoryManager:
    def __init__(self):
        # Dictionary for O(1) lookups: sku -> attributes
        self.products = {}
        # Set for tracking unique categories
        self.categories = set()
        # List of tuples for historical audit logs
        self.transaction_history = []

    def add_product(self, sku, name, category, quantity, price):
        self.products[sku] = {
            "name": name,
            "category": category,
            "quantity": int(quantity),
            "price": float(price)
        }
        self.categories.add(category)
        # Tuple serves as an immutable transaction snapshot
        self.transaction_history.append((sku, "INITIAL_STOCK", int(quantity)))

    def process_transaction(self, sku, delta, transaction_type):
        if sku not in self.products:
            raise KeyError(f"Product reference {sku} missing from catalog maps.")

        self.products[sku]["quantity"] += delta
        self.transaction_history.append((sku, transaction_type, delta))
```

5. Workspace Inventory State Mockups

Primary Hash Catalog (`self.products` State Representation)

```
{  
  "SKU-4021": {"name": "Enterprise Routing Unit", "category": "Networking", "quantity": 14,  
  "SKU-8891": {"name": "Optic Transceiver Module", "category": "Networking", "quantity": 45,  
  "SKU-1092": {"name": "Server Enclosure Unit", "category": "Server Hardware", "quantity":  
}
```

Deduplicated Domains Matrix (`self.categories` State Representation)

```
{"Networking", "Server Hardware"}
```

6. Learning Outcomes

- **Structural Selection Mastery:** Evaluated runtime space-time tradeoffs to choose appropriate built-in types instead of managing arbitrary search structures.
- **Data Integrity Enforcement:** Handled operational states securely by sealing permanent transaction vectors into immutable tuples to prevent data mutations.
- **Optimized Grouping Techniques:** Used sets to extract distinct tracking parameters dynamically, removing the computational cost of iterative lookups.
- **Complex Object Aggregation:** Constructed nested collection schemes (dictionaries containing mixed strings, floats, and integer fields) to handle complex enterprise records.